

STUDIO DELLE TECNICHE DI CLASSIFICAZIONE DI GENERI MUSICALI – A.A. 2023/2024

Mattero Romeo e Davide Orienti

535005 - 533107

Indice Generale

1	Introduzione	2
2	Capitolo 1: Studio del Dataset	2
3	Studio delle Features	3
3.1	Selezione delle features	4
3.1.1	Correlazione tra features	4
3.1.2	Features importance	5
3.1.3	Eliminazione delle feature	6
3.2	Trasformazione delle features	6
4	Tecniche di machine learning	6
4.1	Logistic regression	7
4.1.1	Test effettuati con features di 30 secondi	8
4.1.2	Ottimizzazione degli iperparametri	8
5	Confronto tra dataset da 30 secondi e 3 secondi	9
5.1	Analisi delle Performance	9
5.2	Possibili spiegazioni delle differenze osservate	10
6	Analisi dei Risultati della Regressione Logistica	10
6.1	Effetto della Standardizzazione	10
6.2	Effetto della Grid Search	10
6.3	Conclusioni	11
6.4	Raccomandazioni future	11
7	CAPITOLO 3.2: Random forest	11
7.1	CAPITOLO 3.2.1 TEST EFFETTUATI CON FEATURE DI 30 SECONDI:	11
8	Ottimizzazione degli iperparametri	12
8.1	Parametri ottimizzati	12

1 Introduzione

Questo progetto si basa sullo studio dell'abilità di un sistema di Machine Learning di identificare e classificare il genere musicale di una traccia audio. Il dataset utilizzato per questo task è il noto dataset GTZAN (<https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>), composto da dieci generi musicali distinti, ciascuno composto da 100 tracce audio. I generi musicali che si trovano all'interno di questo dataset sono: blues, classical, country, hiphop, disco, jazz, metal, rock, pop e reggae. Ogni traccia audio è della durata di circa 30 secondi e rappresenta una composizione tipica del genere a cui appartiene, rendendo il dataset ideale per le applicazioni di classificazione. Nel presente studio, verrà analizzata e confrontata l'efficacia di diverse tecniche di estrazione delle feature e di modelli di classificazione, confrontando i risultati ottenuti in termini di accuratezza e robustezza. Come prima analisi si è andato a studiare la distribuzione e caratteristiche dei dati all'interno del dataset. Dopodichè sono state provate e valutate alcune tecniche di feature engineering come creazione di nuove features, selezione di features e trasformazioni di features. Dopo aver effettuato queste tecniche sono stati provate alcune delle tecniche studiate durante il corso e confrontati i risultati ottenuti. Infine è stata sviluppata una rete neurale per tentare di migliorare i dati ottenuti con l'utilizzo di tecniche di machine learning

2 Capitolo 1: Studio del Dataset

Il dataset scelto per questo progetto, noto come GTZAN(<https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>), rappresenta una delle risorse più rilevanti e consolidate nell'ambito della classificazione musicale, fornendo un'ampia gamma di dati utili per l'analisi dei generi musicali. Questo dataset è strutturato in dieci generi musicali: blues, classical, country, hiphop, disco, jazz, metal, rock, pop e reggae. Ogni genere contiene esattamente 100 file audio della durata di 30 secondi ciascuno, garantendo una struttura bilanciata e uniforme che facilita il processo di analisi e interpretazione. L'uniformità del dataset riduce il rischio di sbilanciamenti tra le classi, un aspetto che potrebbe influire negativamente sui risultati di classificazione. Oltre ai file audio, GTZAN contiene ulteriori cartelle con gli spettrogrammi estratti e un insieme di features derivate dai file audio originali. Queste componenti aggiuntive permettono di approcciare il dataset da diverse prospettive analitiche. Per la prima fase del progetto, l'attenzione si è focalizzata sulle features estratte, utilizzando queste come base per costruire modelli di classificazione. La fase successiva ha incluso l'utilizzo degli spettrogrammi e dei file audio originali per sviluppare e addestrare una rete neurale, con l'obiettivo di migliorare le capacità di classificazione e valutare le prestazioni del modello su diverse rappresentazioni dei dati. Un'ulteriore analisi della composizione del dataset ha evidenziato una distribuzione uniforme tra i generi: ciascun genere è rappresentato esattamente da 100 campioni della stessa durata, aspetto che assicura l'assenza di un sovra- o sottorappresentazione di un particolare genere. Oltre ai dati di 30 secondi, GTZAN include anche una rappresentazione dei campioni in formato ridotto. In questo file, ogni traccia è stata suddivisa in dieci segmenti di 3 secondi ciascuno, portando il numero totale dei campioni a 10.000 e incrementando la granularità del dataset. Tale suddivisione permette di aumentare il numero complessivo dei dati forniti al modello, aspetto che può influire positivamente sulle prestazioni complessive della classificazione. Poiché il dataset è etichettato in modo completo, si è scelto un approccio di apprendi-

mento supervisionato. Questo approccio permette di sfruttare al massimo le informazioni presenti nelle etichette, supportando il modello nella distinzione tra i generi musicali.

Table 1: Caratteristiche del dataset GTZAN

Caratteristica	Descrizione
Nome del Dataset	GTZAN (Music Genre Classification Dataset).
Generi Musicali	10 generi: blues, classical, country, hiphop, disco, jazz, metal, rock, pop, reggae.
Numero di Campioni	100 file per genere, ciascuno della durata di 30 secondi.
Bilanciamento	Dataset bilanciato: 1.000 campioni totali, 100 per ciascun genere.
Features Estratte	Contiene features audio e spettrogrammi derivati dai file originali.
Operazione di Segmentazione	Ogni traccia è suddivisa in 10 segmenti da 3 secondi, per un totale di 10.000 campioni.
Rappresentazioni Dati	File audio originali, spettrogrammi, e features estratte.
Tecnica di Apprendimento	Approccio supervisionato utilizzando le etichette di genere.
Scopo	Analisi e classificazione dei generi musicali.

3 Studio delle Features

Il dataset GTZAN include un insieme di 60 **features** estratte dai file audio, ciascuna delle quali fornisce una rappresentazione specifica delle caratteristiche acustiche dei brani. Queste *features* sono state scelte per facilitare il modello di apprendimento nella distinzione tra i vari generi musicali. Di seguito una panoramica delle *features* contenute nel dataset:

- **filename**: Nome del file audio, utilizzato principalmente per identificare il campione e non rilevante per la classificazione.
- **length**: Durata del file audio in secondi.
- **chroma_stft_mean** e **chroma_stft_var**: Media e varianza della trasformata di Fourier con componenti cromatiche, utili per l'analisi delle tonalità e degli accordi.
- **rms_mean** e **rms_var**: Media e varianza del Root Mean Square (RMS) dell'ampiezza, indicano l'intensità sonora media.
- **spectral_centroid_mean** e **spectral_centroid_var**: Media e varianza del centroide spettrale; un valore elevato indica una predominanza di frequenze alte.
- **spectral_bandwidth_mean** e **spectral_bandwidth_var**: Media e varianza della larghezza di banda spettrale, che indica l'ampiezza della gamma di frequenze attorno al centroide spettrale.
- **rolloff_mean** e **rolloff_var**: Media e varianza della frequenza di roll-off, ossia il punto in cui si accumula l'85% dell'energia spettrale.

- **zero_crossing_rate_mean** e **zero_crossing_rate_var**: Media e varianza della frequenza di attraversamento dello zero, utili per distinguere tra suoni rumorosi e tonalità più morbide.
- **harmony_mean** e **harmony_var**: Media e varianza dell'armonia, descrivono la componente armonica del suono e sono cruciali per l'identificazione delle tonalità.
- **percepctr_mean** e **percepctr_var**: Media e varianza del contrasto perceptrale, utilizzato per distinguere le strutture armoniche dal rumore.
- **tempo**: Velocità del brano espressa in battiti per minuto (BPM), utile nella classificazione di generi ritmici.
- **mfcc1_mean - mfcc20_mean** e **mfcc1_var - mfcc20_var**: Media e varianza dei primi 20 coefficienti Mel-Frequency Cepstral Coefficients (MFCC), che rappresentano la forma spettrale del segnale audio su una scala non lineare che riflette la percezione umana delle frequenze. I MFCC sintetizzano informazioni cruciali sull'intonazione e sul timbro del brano e sono fondamentali per la classificazione.
- **label**: Etichetta che identifica il genere musicale del file audio, utilizzata come variabile target per il modello.

Queste *features* forniscono una rappresentazione completa dei file audio, coprendo sia aspetti spettrali (come le frequenze e l'intensità) sia aspetti temporali (come il ritmo e l'armonia). Grazie a queste caratteristiche, è possibile distinguere i generi musicali poiché ciascun genere tende a presentare schemi distinti in termini di frequenze dominanti, intensità, ritmo e armoniche.

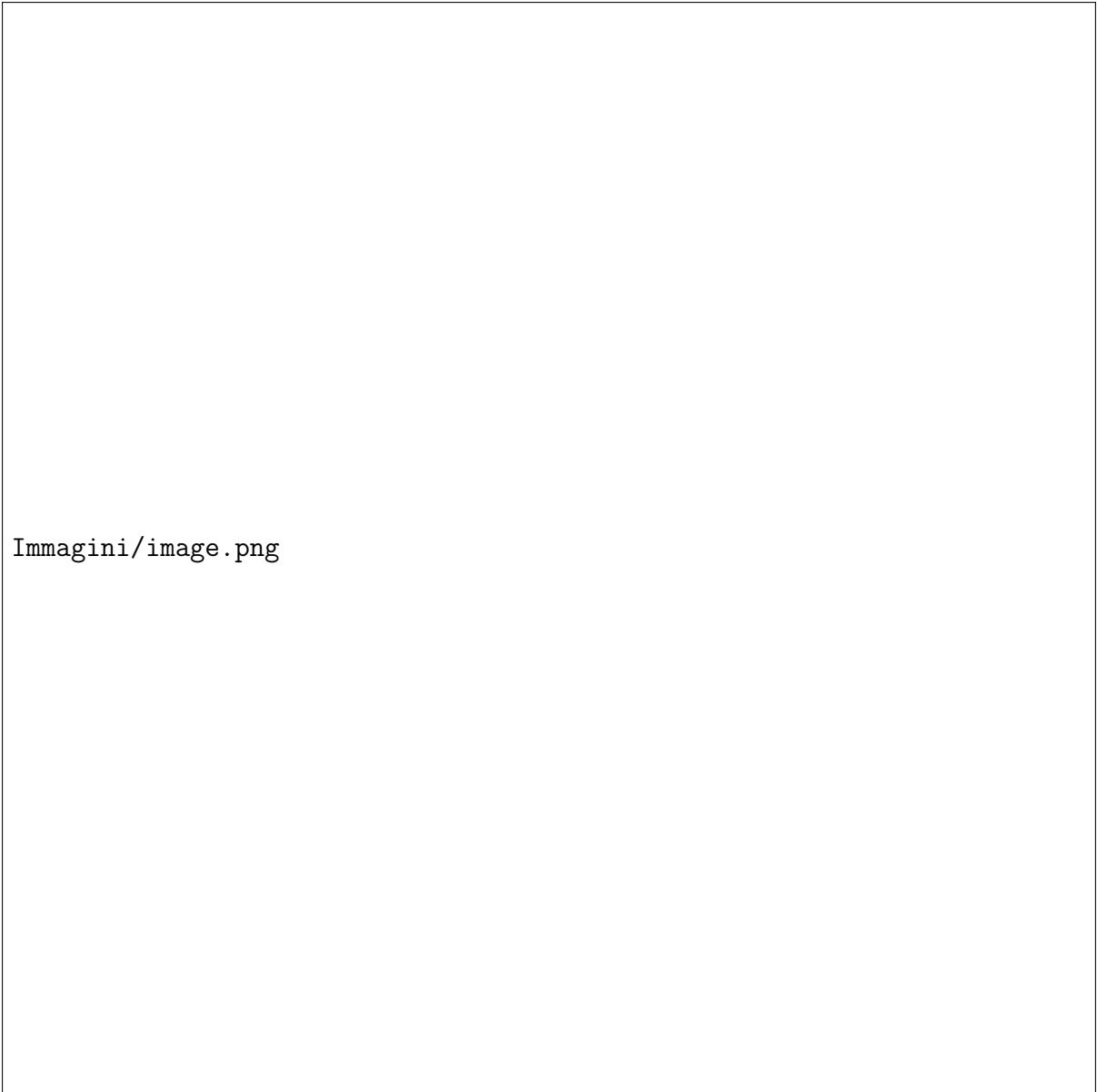
L'uniformità dei campioni di 30 secondi con 100 file per ogni genere garantisce che non vi sia predominanza di alcuna classe. La presenza dei campioni suddivisi in segmenti da 3 secondi, per un totale di 10.000 file, migliora ulteriormente la granularità del dataset. Questo aumento di granularità, inoltre, consente al modello di analizzare le variazioni temporali interne di ciascun file, supportando un'analisi più dettagliata del contenuto sonoro.

Infine, dopo aver analizzato e compreso la struttura delle *features*, abbiamo intrapreso l'applicazione di tecniche di *feature engineering* per ottimizzare ulteriormente le performance del modello e migliorare i risultati di classificazione.

3.1 Selezione delle features

3.1.1 Correlazione tra features

Come prima operazione, ci siamo concentrati sull'analisi delle correlazioni tra le varie features del dataset per identificare eventuali coppie di features con correlazioni elevate. Siamo quindi andati a creare la matrice di correlazione. Studiando i valori della matrice sappiamo che valori prossimi a 1 o a -1 indicano una forte correlazione tra due features, segnalando una relazione lineare diretta o inversa. Quando due features risultano esattamente correlate (valore di correlazione pari a 1 o -1), è possibile eliminare una delle due senza perdita di informazione significativa, semplificando così il modello e riducendo la dimensionalità.



Immagini/image.png

Figure 1: Distribuzione delle 22 query con evidenza per quelle *CPU-intensive* e *Disk-intensive*.

Dall'analisi della matrice di correlazione sono emerse alcune features con elevata correlazione tra loro. Pertanto, abbiamo deciso di testare l'effetto dell'eliminazione di queste features altamente correlate per valutare come tale scelta possa influenzare le prestazioni complessive del modello. I risultati di questo processo saranno testati con delle tecniche di classificazione e discussi nei capitoli successivi.

3.1.2 Features importance

La tecnica della Feature Importance mette in relazione le diverse feature per valutare e interpretare il contributo di ciascuna nelle predizioni del modello. È uno strumento importante per comprendere e ottimizzare il modello, poiché consente di identificare le feature più rilevanti e scartare quelle meno influenti. Questo processo aiuta a ridurre il rumore nei dati e contribuisce a minimizzare la complessità computazionale, migliorando

l'efficienza del modello e la sua capacità di generalizzare.

3.1.3 Eliminazione delle feature

In aggiunta all'analisi delle correlazioni, abbiamo implementato la tecnica di Recursive Feature Elimination (RFE) per selezionare le features più rilevanti. L'RFE è un metodo iterativo di selezione delle features che utilizza un modello di base per valutare l'importanza di ogni caratteristica e rimuove gradualmente quelle meno significative, fino a raggiungere il numero di features desiderato. L'applicazione dell'RFE consente di identificare un sottoinsieme ottimale di features, riducendo così la complessità del modello e mantenendo al contempo le caratteristiche più informative per la classificazione. Questo processo mira a migliorare le prestazioni e a ridurre il rischio di overfitting, concentrandosi sulle features più rilevanti per la distinzione tra generi musicali. Nel nostro studio, abbiamo quindi implementato l'RFE per selezionare le migliori features con vari modelli, valutando successivamente l'efficacia del sottoinsieme selezionato in termini di accuratezza e altre metriche di performance, come discusso nei prossimi capitoli.

————— - Guardare Lasso (Random Forest)

3.2 Trasformazione delle features

Come ultima tecnica nell'ambito del feature engineering che abbiamo utilizzato è stata la standardizzazione. Questo processo consiste nel trasformare i dati in modo che abbiano una media pari a zero e una deviazione standard pari a uno. Tale operazione non solo rende le features comparabili tra loro, ma accelera anche la convergenza degli algoritmi di apprendimento, facilitando l'addestramento dei modelli. Nel nostro studio, sono state prese in considerazione quattro diverse tecniche di standardizzazione, ognuna delle quali ha caratteristiche specifiche e può essere più o meno adatta in base al contesto, alla natura dei dati ed al modello utilizzato. Nel nostro caso specifico tra le possibili standardizzazioni Robust Scaler, Min-Max Scaler, Power Transformer abbiamo scelto di usare Standard Scaler anche come Z-score normalization. Questo approccio è efficace quando i dati seguono una distribuzione normale.

Abbiamo notato immediatamente dei miglioramenti ..

4 Tecniche di machine learning

Nella fase attuale del progetto, ci siamo focalizzati sull'analisi delle varie tecniche di *machine learning* esaminate durante il corso, confrontando le loro performance. In particolare, abbiamo effettuato un confronto tra gli algoritmi utilizzati con il dataset composto da campioni audio della durata di 30 secondi e quelli da 3 secondi. Questa analisi comparativa ci ha permesso di valutare in modo più approfondito l'impatto della granularità dei dati sulle prestazioni degli algoritmi di classificazione.

Per ogni tecnica di *machine learning* adottata, abbiamo estrapolato diverse metriche di valutazione, tra cui:

- **Precision:** Questo parametro misura la proporzione di esempi correttamente classificati come positivi rispetto al totale degli esempi classificati come positivi. In altre parole, la precisione indica quanto i risultati positivi del modello siano accurati. È

calcolata con la formula:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall (o Sensibilità):** Questo parametro misura la proporzione di esempi positivi correttamente identificati rispetto al totale degli esempi realmente positivi. Il *recall* è fondamentale quando è più importante catturare il maggior numero possibile di casi positivi, anche a costo di aumentare il numero di falsi positivi. È calcolato come segue:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

- **Accuracy (Accuratezza):** Questa metrica rappresenta la proporzione di previsioni corrette (sia positive che negative) effettuate dal modello rispetto al numero totale di previsioni. È calcolata come:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Predictions}}$$

- **L1 e L2 (Regularization):** Questi termini si riferiscono a tecniche di regolarizzazione utilizzate per prevenire l'*overfitting* nei modelli di *machine learning*.
 - **L1 Regularization (Lasso Regression):** Questa tecnica aggiunge un termine di penalità alla funzione obiettivo, che è proporzionale alla somma dei valori assoluti dei coefficienti. Favorisce la sparsità nei modelli, riducendo a zero i coefficienti di alcune *features*, il che significa che alcune caratteristiche vengono effettivamente escluse dal modello. Questo è particolarmente utile in situazioni in cui si sospetta che alcune *features* siano irrilevanti.
 - **L2 Regularization (Ridge Regression):** A differenza della regolarizzazione L1, la regolarizzazione L2 penalizza la somma dei quadrati dei coefficienti. Questo approccio tende a ridurre l'ampiezza dei coefficienti senza escludere completamente alcune *features*. È utile quando si desidera mantenere tutte le variabili nel modello, ma si vuole comunque mitigare il rischio di *overfitting*.

Come prima cosa, creiamo un'area dove effettueremo la creazione del nostro *training set* e dello *split*. **(Inserire spiegazione qui).**

4.1 Logistic regression

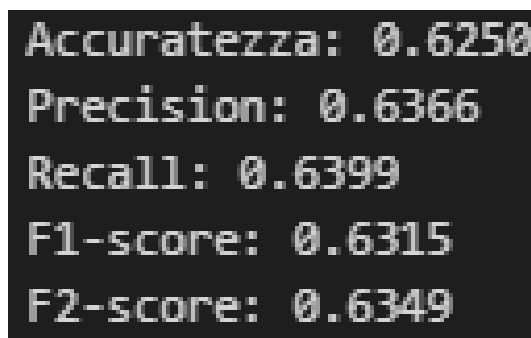
Come primo approccio alla classificazione musicale, è stata scelta la logistic regression per le sue buone prestazioni, semplicità di utilizzo e bassa complessità computazionale. La regressione logistica è particolarmente indicata per problemi di classificazione offrendo così una robusta base per il nostro obiettivo. Prima abbiamo testato e valutato la tecnica con il folder composto da tracce audio di 30 secondi e dopodiché quelli con 3 secondi e confrontato i risultati.

4.1.1 Test effettuati con features di 30 secondi

Come primo passo abbiamo caricato il dataset con i file csv che rappresentano i campioni di 30 secondi.

—foto

Dopodiché abbiamo proceduto con l'esecuzione dell'algoritmo e analizzato i risultati:



```
Accuratezza: 0.6250
Precision: 0.6366
Recall: 0.6399
F1-score: 0.6315
F2-score: 0.6349
```

Figure 2: Distribuzione delle 22 query con evidenza per quelle *CPU-intensive* e *Disk-intensive*.

Nel complesso, i risultati mostrano che il modello ha raggiunto prestazioni stabili e coerenti, con valori di metriche che non si discostano significativamente l'uno dall'altro andando ad escludere comportamenti come underfitting o overfitting. Questo suggerisce un modello ben bilanciato e relativamente robusto.

4.1.2 Ottimizzazione degli iperparametri

Dopo aver studiato i risultati, per migliorarli, si è pensato di procedere andando ad ottimizzare gli iperparametri. La tecnica di ricerca utilizzata è stata la Grid Search. E' stata scelta la grid search perché esamina tutte le combinazioni possibili dei valori specificati per ciascun iperparametro. Assicura di individuare l'insieme di iperparametri ottimale all'interno dello spazio di ricerca, ma può essere lenta su dataset di grandi dimensioni o con molti iperparametri. Dopo aver definito il set di iperparametri da esplorare e individuato quelli ottimali, abbiamo eseguito l'algoritmo.

—SPIEGARE PERCHÉ NEL NOSTRO caso di pochi dati è conveniente

Come si può osservare, i risultati sono ulteriormente migliorati. Questo miglioramento era atteso, poiché la Grid Search identifica la combinazione ottimale di iperparametri, massimizzando le prestazioni del modello. Nel grafico sottostante vengono confrontate le prestazioni di ciascuna tecnica sviluppata, consentendo una valutazione globale delle loro performance.

Dall'analisi del grafico emerge chiaramente che la standardizzazione dei dati, seguita da un'ottimizzazione degli iperparametri tramite Grid Search, conduce a prestazioni ottimali del modello. Di conseguenza, per i prossimi algoritmi verranno adottate le medesime procedure, al fine di ottenere risultati altrettanto ottimizzati.

5 Confronto tra dataset da 30 secondi e 3 secondi

Nel grafico sottostante vengono confrontati i risultati ottenuti utilizzando le features dei dataset con campioni di 30 secondi e di 3 secondi. Si evidenzia che inizialmente il dataset da 3 secondi risulta molto meno preciso rispetto a quello da 30 secondi, ma con l'applicazione di tecniche di ottimizzazione come la GridSearch e il *standardization*, si osserva un netto miglioramento. Questo porta il dataset da 3 secondi a superare leggermente le prestazioni di quello da 30 secondi.

Figure 3: Confronto delle accuratezze tra dataset di 30 secondi e 3 secondi nei tre scenari considerati.

La situazione descritta può essere spiegata analizzando il comportamento dei modelli, la natura del dataset e l'impatto delle tecniche di ottimizzazione come la GridSearch.

5.1 Analisi delle Performance

1. Performance iniziale (senza GridSearch):

- **Dataset di 3 secondi:** 10.000 campioni, Accuratezza = 0.45
 - I dati di 3 secondi catturano solo una piccola finestra temporale dell'audio, che potrebbe non essere sufficiente per rappresentare adeguatamente le caratteristiche discriminanti tra le classi.
 - Tuttavia, il grande numero di campioni (10.000) introduce una potenziale quantità di rumore che confonde il modello.
- **Dataset di 30 secondi:** 1.000 campioni, Accuratezza = 0.65
 - I 30 secondi catturano una rappresentazione più completa del brano, fornendo informazioni temporali più ampie utili per la classificazione.
 - Un numero inferiore di campioni (1.000) rende il dataset meno influenzato dal rumore.

2. Performance dopo GridSearch:

- **Dataset di 3 secondi:** Accuratezza migliorata (superiore a quella del dataset da 30 secondi)
 - La GridSearch ha ottimizzato i parametri del modello (ad esempio, *C*, *penalty*, *solver*), migliorandone l'adattamento al dataset.
 - La grande quantità di campioni (10.000) ha permesso una migliore generalizzazione.
- **Dataset di 30 secondi:** Accuratezza inferiore
 - I dati più lunghi introducono complessità temporale, con possibili informazioni irrilevanti o ridondanti.
 - La minore quantità di campioni (1.000) limita l'efficacia dell'ottimizzazione.

5.2 Possibili spiegazioni delle differenze osservate

- **Quantità di dati:** Il dataset da 3 secondi beneficia di un numero maggiore di campioni, favorendo l'apprendimento e la generalizzazione.
- **Rumore nei dati:** I 30 secondi possono includere parti di audio meno significative (ad esempio, silenzio o rumore di fondo), mentre i segmenti più corti catturano elementi più rappresentativi.
- **Effetti della GridSearch:** L'ottimizzazione dei parametri per il dataset da 3 secondi ha compensato le limitazioni iniziali, migliorando significativamente le prestazioni.
- **Bias e varianza:** Il dataset da 3 secondi potrebbe avere una varianza più bassa, rendendo il problema più semplice da risolvere rispetto ai dati da 30 secondi, che introducono maggiore complessità.

6 Analisi dei Risultati della Regressione Logistica

6.1 Effetto della Standardizzazione

L'applicazione della regressione logistica sui dati relativi a 30 secondi ha evidenziato un significativo miglioramento dell'accuratezza passando da 0.62 (senza standardizzazione) a 0.74 (con standardizzazione). Questo incremento è attribuibile alla sensibilità della regressione logistica alle scale delle feature. La standardizzazione, riportando tutte le feature su una scala comune (media 0 e deviazione standard 1), consente al modello di convergere più rapidamente verso una soluzione ottimale, evitando che feature con valori più elevati dominino il processo di apprendimento.

6.2 Effetto della Grid Search

Nonostante l'implementazione della tecnica di ottimizzazione degli iperparametri `*grid search*`, l'accuratezza del modello non ha registrato ulteriori incrementi, rimanendo stabile a 0.74. Questa osservazione può essere riconducibile a diversi fattori:

- **Ottimalità dei parametri iniziali:** I parametri predefiniti della regressione logistica, combinati con la standardizzazione, potrebbero già essere prossimi al punto di ottimo per il dataset in esame.
- ****Limiti del modello lineare:**** La regressione logistica, essendo un modello lineare, potrebbe non essere in grado di catturare relazioni non lineari presenti nei dati.
- ****Qualità del dataset:**** La presenza di rumore, correlazioni deboli tra le feature e le classi, o altre caratteristiche intrinseche del dataset potrebbero limitare le potenzialità di miglioramento del modello.
- ****Scelta della metrica di valutazione:**** L'accuracy, pur essendo una metrica comune, potrebbe non essere sufficientemente sensibile a piccole variazioni nelle prestazioni del modello, specialmente in presenza di dataset bilanciati.

6.3 Conclusioni

L'analisi condotta evidenzia l'importanza della standardizzazione delle feature nell'applicazione della regressione logistica. Tuttavia, la **grid search** non ha portato a ulteriori miglioramenti, suggerendo che:

- I parametri del modello erano già ottimizzati per il dataset in esame.
- La regressione logistica potrebbe non essere il modello più adatto a catturare le complessità dei dati.

Per ottenere ulteriori miglioramenti, si suggerisce di esplorare:

- ****Modelli più complessi:**** Come le **support vector machine** (SVM) o le **random forest**, in grado di gestire relazioni non lineari.
- ****Tecniche di preprocessing avanzate:**** Come l'ingegnerizzazione delle feature o la riduzione della dimensionalità, per estrarre informazioni più rilevanti dai dati.

6.4 Raccomandazioni future

1. Analizzare la distribuzione delle *features* per individuare eventuali rumori o ridondanze nel dataset da 30 secondi.
2. Segmentare i campioni da 30 secondi in intervalli più piccoli (ad esempio, 3 secondi) per bilanciare quantità e qualità dei dati.
3. Esplorare modelli più complessi, come le reti neurali convoluzionali, per catturare meglio la complessità temporale del dataset da 30 secondi.

7 CAPITOLO 3.2: Random forest

Random Forest è un algoritmo di apprendimento supervisionato che combina molti alberi decisionali per fare previsioni. Questo metodo è particolarmente efficace per la classificazione perché tende a essere robusto agli outlier. Quindi per il nostro task potrebbe essere una buona soluzione alternativa.

7.1 CAPITOLO 3.2.1 TEST EFFETTUATI CON FEATURE DI 30 SECONDI:

Partiamo andando a caricare il dataset con i capini di 30 secondi come fatto precedentemente. A differenza della tecnica della Logistic Regression vista precedentemente si è optato di non andare a standardizzare. Non sono stati standardizzati per via della struttura e del funzionamento di questa tecnica. Gli alberi decisionali funzionano con soglie di valore per suddividere i dati, quindi non sono influenzati dal fatto che le variabili siano su scale diverse. Questa tecnica sfrutta un insieme di alberi indipendenti, e ciascun albero valuta i dati in base alla posizione relativa dei valori, non in base alla loro grandezza. Andando a eseguire una prima volta per capire il funzionamento ed abbiamo avuto i seguenti risultati:

OTTIMIZZAZIONE IPERPARAMETRI

Per migliorare i risultati della prima esecuzione, si è deciso di ottimizzare gli iperparametri del modello. A tal fine, è stata utilizzata la tecnica della *Grid Search*, che esplora sistematicamente tutte le combinazioni di iperparametri specificati, individuando quella che ottimizza le prestazioni del modello.

8 Ottimizzazione degli iperparametri

Per migliorare i risultati ottenuti nella prima esecuzione, si è deciso di ottimizzare gli iperparametri del modello. A tal fine, è stata utilizzata la tecnica della *Grid Search*, che consente di esplorare sistematicamente tutte le combinazioni degli iperparametri specificati, individuando quella che ottimizza le prestazioni del modello.

8.1 Parametri ottimizzati

I parametri selezionati per l'ottimizzazione tramite *Grid Search* sono i seguenti:

- **n_estimators**: Numero di alberi nella foresta.
- **max_depth**: Profondità massima di ciascun albero.
- **min_samples_split**: Numero minimo di campioni richiesti per suddividere un nodo.
- **min_samples_leaf**: Numero minimo di campioni richiesti per essere considerati una foglia.

L'applicazione della *Grid Search* ha consentito di esplorare combinazioni diverse di questi iperparametri, migliorando significativamente le prestazioni del modello.

A seguito dell'esecuzione della *Grid Search*, sono stati individuati questi iperparametri come i più adatti per il nostro task.

— IMG

Utilizzando gli iperparametri ottimali individuati, è stato eseguito il modello ottenendo i seguenti risultati:

article booktabs

Table 2: Confronto tra i modelli

Metodo	Accuratezza
Modello base	76%
Modello ottimizzato con Grid Search	80%